

University of Moratuwa
Department of Electronic and Telecommunication



EN2111 FPGA Assignment
UART Implementation in FPGA

Group No – 37

Member 1 Weerakoon A.H.T.M. Index No: 230689A
Member 2 Weerasinghe J.A.H.R. Index No: 230697V
Member 3 Wijesekara W.A.G.S. Index No: 230724E

May 1, 2026

Contents

1	Introduction	2
2	RTL Design and Architecture	2
2.1	UART Transceiver RTL Code	2
3	Simulation Verification	5
3.1	Testbench Code	6
4	FPGA Hardware Implementation	7
4.1	Pin Assignment	7
4.2	Top Module FPGA Code	7
4.3	Hardware Verification	9
4.4	Oscilloscope Waveform	10
5	Observations and Discussion	11
6	Conclusion	11

1 Introduction

The Universal Asynchronous Receiver-Transmitter (UART) is a widely utilized protocol for asynchronous serial communication between devices. By eliminating the need for a shared synchronization clock signal, UART significantly simplifies hardware design and implementation. The boundaries of a data stream are represented by specific start and stop bits appended to the data payload. Because there is no shared clock line, both the transmitting and receiving devices must be pre-configured to operate at the exact same speed, known as the baud rate. Communication typically occurs over just two data lines—a Transmit (TX) line and a Receive (RX) line. When the connection is idle, the line is held at a logic HIGH. To initiate a transfer, the transmitter pulls the line LOW for one clock cycle as a start bit, signaling the receiver to wake up and begin reading. This is immediately followed by the data frame, which is usually a chunk of 8 bits sent sequentially, alongside an optional parity bit for low-level error checking. Finally, the transmitter pulls the line back to HIGH for at least one clock cycle to serve as the stop bit, successfully completing the packet and returning the line to its idle state, ready for the next transmission.

2 RTL Design and Architecture

The UART Transceiver is a parameterized, full-duplex module that utilizes dedicated finite state machines for serial data transmission and reception. It manages clock constraints internally by using counters derived from the system clock to precisely generate the target 115200 baud rate without needing external dividers. The design strictly adheres to standard RS-232 protocol specifications by implementing proper start and stop bit framing, least-significant-bit-first data transfer, a double-flop synchronizer to prevent metastability on asynchronous inputs, and robust middle-of-bit sampling to ensure accurate data recovery.

2.1 UART Transceiver RTL Code

```
1 module uart_transceiver(  
2     input wire clk,           // System clock (50 MHz default)  
3     input wire rst_n,        // Active low reset  
4     // UART TX Interface  
5     input wire tx_start,     // Start transmission  
6     input wire [7:0] tx_data, // Data to transmit  
7     output reg tx_busy,      // Transmitter busy flag  
8     output reg tx,           // Serial output  
9     // UART RX Interface  
10    input wire rx,            // Serial input  
11    output reg rx_done,       // Reception complete flag  
12    output reg [7:0] rx_data // Received parallel data  
13 );  
14  
15  
16 // Parameters  
17 parameter CLK_FREQ = 50_000_000;  
18 parameter BAUD_RATE = 115200;  
19 parameter CLKS_PER_BIT = CLK_FREQ / BAUD_RATE;  
20
```

```

21 // State definitions
22 localparam IDLE = 2'b00;
23 localparam START = 2'b01;
24 localparam DATA = 2'b10;
25 localparam STOP = 2'b11;
26
27 // TX logic registers
28 reg [1:0] tx_state;
29 reg [15:0] tx_clk_count;
30 reg [2:0] tx_bit_count;
31 reg [7:0] tx_shift_reg;
32
33 // RX logic registers
34 reg [1:0] rx_state;
35 reg [15:0] rx_clk_count;
36 reg [2:0] rx_bit_count;
37 reg [7:0] rx_shift_reg;
38 reg rx_d1, rx_d2; // Double-flop synchronizer
39
40 // Synchronize RX input to avoid metastability
41 always @(posedge clk or negedge rst_n) begin
42     if (!rst_n) begin
43         rx_d1 <= 1'b1;
44         rx_d2 <= 1'b1;
45     end else begin
46         rx_d1 <= rx;
47         rx_d2 <= rx_d1;
48     end
49 end
50
51 // TX State Machine
52 always @(posedge clk or negedge rst_n) begin
53     if (!rst_n) begin
54         tx_state <= IDLE;
55         tx_clk_count <= 0;
56         tx_bit_count <= 0;
57         tx_busy <= 0;
58         tx <= 1'b1;
59         tx_shift_reg <= 8'h00;
60     end else begin
61         case (tx_state)
62             IDLE: begin
63                 tx <= 1'b1;
64                 tx_clk_count <= 0;
65                 tx_bit_count <= 0;
66                 if (tx_start && !tx_busy) begin
67                     tx_busy <= 1'b1;
68                     tx_shift_reg <= tx_data;
69                     tx_state <= START;
70                 end
71             end
72             START: begin
73                 tx <= 1'b0;
74                 if (tx_clk_count < CLKS_PER_BIT - 1)
75                     tx_clk_count <= tx_clk_count + 1'b1;
76                 else begin
77                     tx_clk_count <= 0;
78                     tx_state <= DATA;

```

```

79         end
80     end
81     DATA: begin
82         tx <= tx_shift_reg[0];
83         if (tx_clk_count < CLKS_PER_BIT - 1)
84             tx_clk_count <= tx_clk_count + 1'b1;
85         else begin
86             tx_clk_count <= 0;
87             if (tx_bit_count < 7) begin
88                 tx_bit_count <= tx_bit_count + 1'b1;
89                 tx_shift_reg <= {1'b0, tx_shift_reg[7:1]};
90             end else begin
91                 tx_bit_count <= 0;
92                 tx_state <= STOP;
93             end
94         end
95     end
96     STOP: begin
97         tx <= 1'b1;
98         if (tx_clk_count < CLKS_PER_BIT - 1)
99             tx_clk_count <= tx_clk_count + 1'b1;
100        else begin
101            tx_clk_count <= 0;
102            tx_busy <= 1'b0;
103            tx_state <= IDLE;
104        end
105    end
106    default: tx_state <= IDLE;
107 endcase
108 end
109 end
110
111 // RX State Machine
112 always @(posedge clk or negedge rst_n) begin
113     if (!rst_n) begin
114         rx_state <= IDLE;
115         rx_clk_count <= 0;
116         rx_bit_count <= 0;
117         rx_done <= 0;
118         rx_data <= 8'h00;
119         rx_shift_reg <= 8'h00;
120     end else begin
121         if (rx_done) rx_done <= 1'b0;
122         case (rx_state)
123             IDLE: begin
124                 rx_clk_count <= 0;
125                 rx_bit_count <= 0;
126                 if (rx_d2 == 1'b0) rx_state <= START;
127             end
128             START: begin
129                 if (rx_clk_count == (CLKS_PER_BIT - 1) / 2) begin
130                     if (rx_d2 == 1'b0) begin
131                         rx_clk_count <= rx_clk_count + 1'b1;
132                     end else begin
133                         rx_state <= IDLE;
134                     end
135                 end else if (rx_clk_count < CLKS_PER_BIT - 1) begin
136                     rx_clk_count <= rx_clk_count + 1'b1;

```

```

137         end else begin
138             rx_clk_count <= 0;
139             rx_state <= DATA;
140         end
141     end
142     DATA: begin
143         if (rx_clk_count == (CLKS_PER_BIT - 1) / 2) begin
144             rx_shift_reg <= {rx_d2, rx_shift_reg[7:1]};
145             rx_clk_count <= rx_clk_count + 1'b1;
146         end else if (rx_clk_count < CLKS_PER_BIT - 1) begin
147             rx_clk_count <= rx_clk_count + 1'b1;
148         end else begin
149             rx_clk_count <= 0;
150             if (rx_bit_count < 7)
151                 rx_bit_count <= rx_bit_count + 1'b1;
152             else begin
153                 rx_bit_count <= 0;
154                 rx_state <= STOP;
155             end
156         end
157     end
158     STOP: begin
159         if (rx_clk_count < CLKS_PER_BIT - 1)
160             rx_clk_count <= rx_clk_count + 1'b1;
161         else begin
162             rx_done <= 1'b1;
163             rx_data <= rx_shift_reg;
164             rx_state <= IDLE;
165         end
166     end
167     default: rx_state <= IDLE;
168 endcase
169 end
170 end
171 endmodule

```

Listing 1: UART Transceiver Verilog Code

3 Simulation Verification

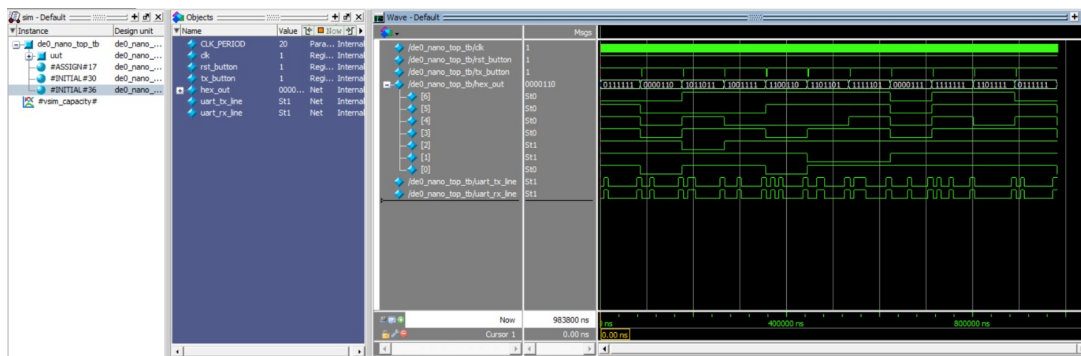


Figure 1: Timing diagram captured during simulation verification.

3.1 Testbench Code

```
1  `timescale 1ns/1ps
2
3  module de0_nano_top_tb();
4
5      // Parameters
6      parameter CLK_PERIOD = 20; // 50 MHz clock
7
8      // Testbench Signals
9      reg clk;
10     reg rst_button; // Simulates KEY0
11     reg tx_button; // Simulates KEY1
12     wire [6:0] hex_out;
13     wire uart_tx_line;
14     wire uart_rx_line;
15
16     // Loopback: Connect TX directly to RX
17     assign uart_rx_line = uart_tx_line;
18
19     // Instantiate the Top-Level Module
20     de0_nano_top uut (
21         .CLOCK_50(clk),
22         .KEY0(rst_button),
23         .KEY1(tx_button),
24         .HEX0(hex_out),
25         .GPIO_0_0(uart_tx_line),
26         .GPIO_0_1(uart_rx_line)
27     );
28
29     // Clock Generation
30     initial begin
31         clk = 0;
32         forever #(CLK_PERIOD/2) clk = ~clk;
33     end
34
35     // Simulation Procedure
36     initial begin
37         // Initialize Signals
38         rst_button = 0; // Press Reset (Active Low)
39         tx_button = 1; // Button not pressed (Active Low)
40
41         // 1. Apply Reset
42         #(CLK_PERIOD * 10);
43         rst_button = 1; // Release Reset
44         #(CLK_PERIOD * 10);
45
46         // 2. Perform 3 Increments (0 -> 1 -> 2)
47         // We will repeat this 3 times to see the count go up
48         repeat (11) begin
49             $display("--- Pressing KEY1 to increment ---");
50
51             // Press the button
52             tx_button = 0;
53             #(CLK_PERIOD * 20); // Hold for a few cycles
54
55             // Release the button
56             tx_button = 1;
```

```

57
58         // Wait for the UART transmission and reception to finish
59         // At 115200 baud, one byte takes ~86,000ns.
60         // We wait slightly longer to be safe.
61         #89000;
62
63         $display("Current 7-segment pattern: %b", hex_out);
64     end
65
66     // 3. Test the wrap-around (Skip to 9)
67     // Manually forcing the internal tx_data to 8 would be hard,
68     // so we just run the simulation long enough to see the
69     patterns.
70
71     $display("Simulation finished.");
72     $stop; // Pause simulation
73
74 endmodule

```

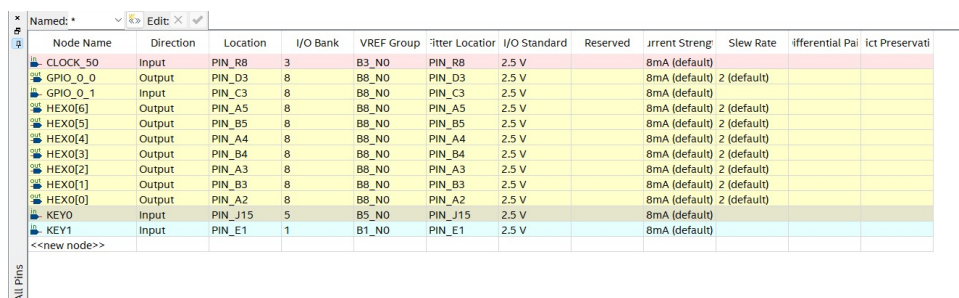
Listing 2: UART Testbench Verilog Code

4 FPGA Hardware Implementation

Once the code was uploaded to the FPGA, pressing KEY 01 transmitted an incrementing number (from 0 to 9). (KEY 02 used for reset) The received number was then output to an external 7-segment display, while its corresponding binary pattern was simultaneously displayed on the onboard LEDs.

4.1 Pin Assignment

The following figure shows the pin assignment configuration used to map the RTL design ports to the physical FPGA pins.



Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Input Preservation
CLOCK_50	Input	PIN_R8	3	B3_NO	PIN_R8	2.5 V		8mA (default)			
GPIO_0_0	Output	PIN_D3	8	B8_NO	PIN_D3	2.5 V		8mA (default)	2 (default)		
GPIO_0_1	Input	PIN_C3	8	B8_NO	PIN_C3	2.5 V		8mA (default)			
HEX0[6]	Output	PIN_A5	8	B8_NO	PIN_A5	2.5 V		8mA (default)	2 (default)		
HEX0[5]	Output	PIN_B5	8	B8_NO	PIN_B5	2.5 V		8mA (default)	2 (default)		
HEX0[4]	Output	PIN_A4	8	B8_NO	PIN_A4	2.5 V		8mA (default)	2 (default)		
HEX0[3]	Output	PIN_B4	8	B8_NO	PIN_B4	2.5 V		8mA (default)	2 (default)		
HEX0[2]	Output	PIN_A3	8	B8_NO	PIN_A3	2.5 V		8mA (default)	2 (default)		
HEX0[1]	Output	PIN_B3	8	B8_NO	PIN_B3	2.5 V		8mA (default)	2 (default)		
HEX0[0]	Output	PIN_A2	8	B8_NO	PIN_A2	2.5 V		8mA (default)	2 (default)		
KEY0	Input	PIN_J15	5	B5_NO	PIN_J15	2.5 V		8mA (default)			
KEY1	Input	PIN_E1	1	B1_NO	PIN_E1	2.5 V		8mA (default)			
<<new node>>											

Figure 2: Quartus pin assignment screenshot.

4.2 Top Module FPGA Code

```

1 module de0_nano_top (
2     input wire CLOCK_50,
3     input wire KEY0,           // Reset (Active Low)
4     input wire KEY1,           // Increment and Transmit trigger
5     output wire [6:0] HEX0,    // 7-segment display segments (a-g)

```

```

6      output wire GPIO_0_0,    // UART TX
7      input wire GPIO_0_1     // UART RX
8  );
9
10     // Internal Wires and Registers
11     wire tx_busy, tx, rx_done, rst_n, key1_pulse;
12     wire [7:0] rx_data;
13     reg tx_start;
14     reg [7:0] tx_data;
15     reg [3:0] key0_debounce, key1_debounce;
16
17     // 1. Reset Debounce Logic (KEY0)
18     always @(posedge CLOCK_50 or negedge KEY0) begin
19         if (!KEY0) key0_debounce <= 4'b0000;
20         else key0_debounce <= {key0_debounce[2:0], 1'b1};
21     end
22     assign rst_n = &key0_debounce;
23
24     // 2. Increment Button Debounce Logic (KEY1)
25     always @(posedge CLOCK_50 or negedge rst_n) begin
26         if (!rst_n) key1_debounce <= 4'b0000;
27         else key1_debounce <= {key1_debounce[2:0], KEY1};
28     end
29     // Detects the falling edge (when the button is pressed)
30     assign key1_pulse = (key1_debounce == 4'b1110);
31
32     // 3. UART Transceiver Instance
33     uart_transceiver #(
34         .CLK_FREQ(50_000_000),
35         .BAUD_RATE(115200)
36         //.BAUD_RATE(115200)
37
38     ) uart_inst (
39         .clk(CLOCK_50),
40         .rst_n(rst_n),
41         .tx_start(tx_start),
42         .tx_data(tx_data),
43         .tx_busy(tx_busy),
44         .tx(tx),
45         .rx(GPIO_0_1),
46         .rx_done(rx_done),
47         .rx_data(rx_data)
48     );
49
50     assign GPIO_0_0 = tx;
51
52     // 4. Counter Logic (0 to 9)
53     always @(posedge CLOCK_50 or negedge rst_n) begin
54         if (!rst_n) begin
55             tx_start <= 1'b0;
56             tx_data <= 8'd0;
57         end else if (key1_pulse && !tx_busy) begin
58             tx_start <= 1'b1;
59             // Check if we reached 9, if so, wrap back to 0
60             if (tx_data >= 8'd9) begin
61                 tx_data <= 8'd0;
62             end else begin
63                 tx_data <= tx_data + 1'b1;

```

```

64         end
65     end else begin
66         tx_start <= 1'b0;
67     end
68 end
69
70 // 5. 7-Segment Decoder Instance
71 // This decodes the RECEIVED data (rx_data) for display
72 bcd_to_7seg display_unit (
73     .bcd(rx_data[3:0]),
74     .seg(HEX0)
75 );
76
77 endmodule
78
79 // 6. BCD to 7-Segment Decoder Module
80 // Designed for Common Anode (0 = ON, 1 = OFF)
81 module bcd_to_7seg (
82     input wire [3:0] bcd,
83     output reg [6:0] seg
84 );
85     always @(*) begin
86         case (bcd)
87             4'h0: seg = 7'b0111111; // 0
88             4'h1: seg = 7'b0000110; // 1
89             4'h2: seg = 7'b1011011; // 2
90             4'h3: seg = 7'b1001111; // 3
91             4'h4: seg = 7'b1100110; // 4
92             4'h5: seg = 7'b1101101; // 5
93             4'h6: seg = 7'b1111101; // 6
94             4'h7: seg = 7'b0000111; // 7
95             4'h8: seg = 7'b1111111; // 8
96             4'h9: seg = 7'b1101111; // 9
97             default: seg = 7'b0000000; // Off
98         endcase
99     end
100 endmodule

```

Listing 3: Top Module FPGA Verilog Code

4.3 Hardware Verification

Initial testing was conducted in a loopback configuration, utilizing a single FPGA as both the transmitter and receiver. Following successful local verification, communication was established between two separate FPGAs, as illustrated below.

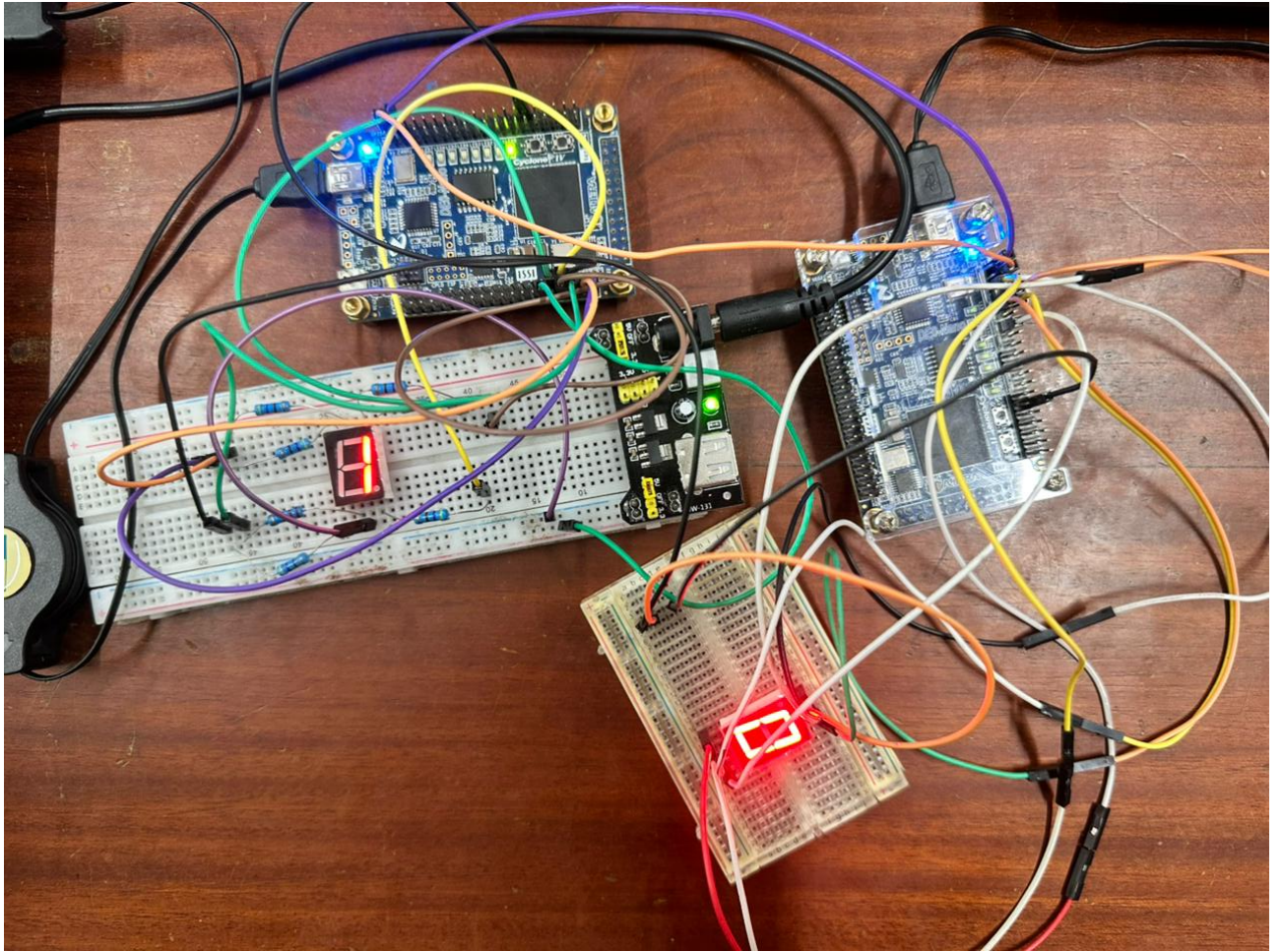


Figure 3: Real-time hardware setup showing communication with another group's FPGA.

4.4 Oscilloscope Waveform

The waveform captured from the FPGA TX/RX lines using an oscilloscope is shown below, verifying the correct baud rate and frame structure.

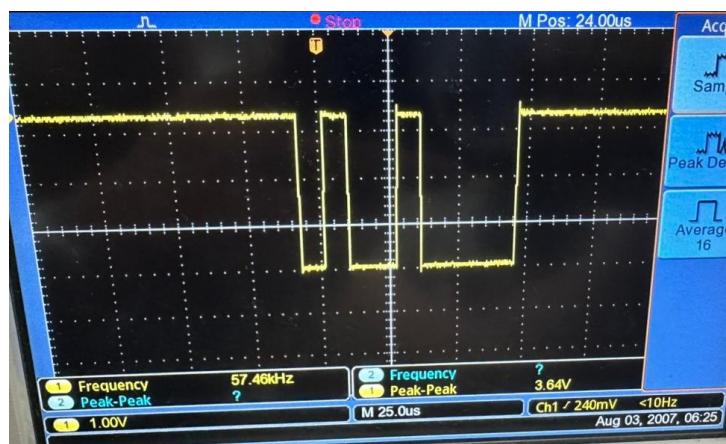


Figure 4: UART waveform from Oscilloscope.

5 Observations and Discussion

Functional verification utilizing the 7-segment displays and onboard LEDs successfully confirmed that the serial data was correctly packetized and depacketized. Further hardware validation was conducted using an oscilloscope with the horizontal timebase set to 25.0 μs per division as above image. Under these settings, the measured width of an individual data bit was approximately 8.7 μs . This directly corroborates the target 115200 baud rate generated by the FPGA (where one bit equals 8.68 μs). Additionally, the measured peak-to-peak amplitude of 3.64V closely aligns with the FPGA's standard 3.3V logic levels, indicating good signal integrity.

6 Conclusion

In conclusion, the UART transceiver design was successfully validated across both theoretical and physical domains. The simulation phase confirmed the logical correctness of the state machines, ensuring that baud rate generation and data framing adhered strictly to protocol specifications. Subsequently, the hardware verification phase proved the design's real-world reliability. Oscilloscope measurements confirmed precise timing and physical signal integrity, while the successful cross-board communication—verified via 7-segment displays and onboard LEDs—demonstrated flawless data packetization and depacketization. Ultimately, the implemented RTL architecture proved to be both logically sound and highly robust for asynchronous serial communication between FPGAs.